



Natural Language Processing

Lecture 5: Applied NLP Tasks [Part I].

Previous topics recap



What is commonly referred to as a logit?





What is representation learning?



In an n-gram language model, we often estimate probabilities like: $P('NLP' \mid 'I', 'love')$. This expression conditions the next word "NLP" on the previous two words "I love."

What type of n-gram model does this represent?





What distinguishes a recurrent neural network (RNN) from a standard feedforward network?





What is the main advantage of Long Short-Term Memory (LSTM) networks over basic RNNs?





The formula for scaled dot-product self-attention is ...





Why do we need to have $\sqrt{d_k}$ in the self-attention denominator





Have you already started your first homework?



Agenda

1. Learning paradigms: supervised, unsupervised, self-supervised, reinforcement learning
2. Text Classification: Sentiment analysis, Toxicity/hate speech detection, Propaganda detection.
3. Text Summarization.
4. Machine Translation.
5. Question Answering.
6. Dialogue & Conversational Systems. Task oriented agents.

Applied NLP Tasks

POS

NER

Topic Modeling

Summarization

Classification

Translation

Information
Extraction

Document
processing/OCR

Conversational

Question
Answering

Applied NLP Tasks

POS

NER

Topic Modeling

Classification

Summarization

Information
Extraction

Translation

Document
processing/OCR

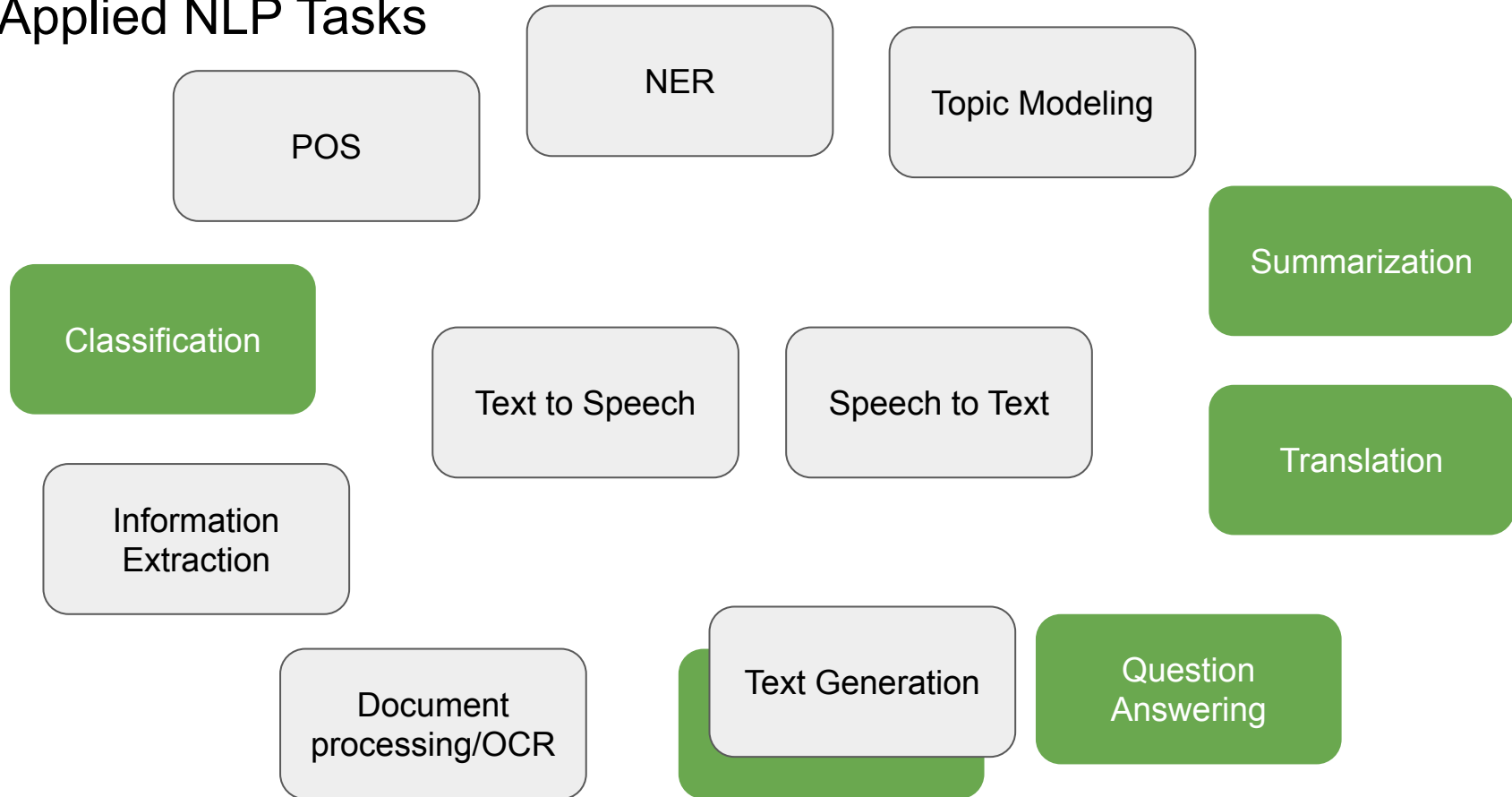
Conversational

Question
Answering

Applied NLP Tasks



Applied NLP Tasks



Supervised learning

Learn mapping $f: X \rightarrow Y$ from labeled pairs (x, y) .

training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

naive Bayes,
support vector machines,
neural networks,
fine-tuned language models, etc.

Supervised learning

Learn mapping $f: X \rightarrow Y$ from labeled pairs (x, y) .

training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

Objective:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

naive Bayes,
support vector machines,
neural networks,
fine-tuned language models, etc.

Supervised learning

Learn mapping $f: X \rightarrow Y$ from labeled pairs (x, y) .

Objective:

training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

Anything else?

naive Bayes,
support vector machines,
neural networks,
fine-tuned language models, etc.

Text classification: spam detection, sentiment analysis, **xxx**

Sequence labeling: POS tagging, Named Entity Recognition

Structured prediction: dependency parsing, **xxx**

Supervised learning

Learn mapping $f: X \rightarrow Y$ from labeled pairs (x, y) .

training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

Problems?

naive Bayes,
support vector machines,
neural networks,
fine-tuned language models, etc.

Text classification: spam detection, sentiment analysis

Sequence labeling: POS tagging, Named Entity Recognition

Structured prediction: dependency parsing, translation

Objective:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

Unsupervised learning

Learn hidden structure from **unlabeled data**

$$\{x_1, x_2, \dots, x_n\}$$

Clustering: k-means, hierarchical, spectral clustering

Representation learning: Word2Vec, GloVe

Dimensionality reduction: PCA, t-SNE, autoencoders

Unsupervised learning

Learn hidden structure from **unlabeled data**

$$\{x_1, x_2, \dots, x_n\}$$

Clustering: k-means, hierarchical, spectral clustering

Representation learning: Word2Vec, GloVe

Dimensionality reduction: PCA, t-SNE, autoencoders

Document/topic discovery (e.g., **news** clustering)

Lexical semantics (word **embeddings**, **semantic similarity**)

Information retrieval (query/document embedding)

Summarization & exploratory analysis (topic grouping)

Unsupervised learning

Learn hidden structure from **unlabeled data**

$$\{x_1, x_2, \dots, x_n\}$$

Clustering: k-means, hierarchical, spectral clustering

Representation learning: Word2Vec, GloVe

Dimensionality reduction: PCA, t-SNE, autoencoders

Application?

Document/topic discovery (e.g., **news** clustering)

Lexical semantics (word **embeddings**, **semantic similarity**)

Information retrieval (query/document embedding)

Summarization & exploratory analysis (topic grouping)

Unsupervised learning

Learn hidden structure from **unlabeled data**

$$\{x_1, x_2, \dots, x_n\}$$

Clustering: k-means, hierarchical, spectral clustering

Representation learning: Word2Vec, GloVe

Dimensionality reduction: PCA, t-SNE

Application?

Document/topic discovery (e.g., **news** clustering)

Lexical semantics (word **embeddings**, **semantic similarity**)

Information retrieval (query/document embedding)

Summarization & exploratory analysis (topic grouping)

Searching for relevant documents, grouping texts/reviews by themes, detecting emerging topics (social networks)

Self-supervised learning

Learn from raw text by creating **pseudo-labels** from the data itself.

$$\min_{\theta} \mathbb{E}_{x \sim D} \ell(f_{\theta}(\tilde{x}), x)$$

Causal Language Modeling (CLM): predict next token – GPT, GPT-2, GPT-3

Masked Language Modeling (MLM): predict masked tokens – BERT

Permutation/Span Prediction: T5

Contrastive Learning: CLIP (text-image)

Self-supervised learning

Learn from raw text by creating **pseudo-labels** from the data itself.

$$\min_{\theta} \mathbb{E}_{x \sim D} \ell(f_{\theta}(\tilde{x}), x)$$

Causal Language Modeling (CLM): predict next token – GPT, GPT-2, GPT-3

Masked Language Modeling (MLM): predict masked tokens – BERT

Permutation/Span Prediction: T5

Contrastive Learning: SimCSE, CLIP (text-image)

“Unlike left-to right language model pre-training, the **MLM** objective enables the representation to fuse the left and the right context, which allows us to pretrain a deep bidirectional Transformer.”

Self-supervised learning

Learn from raw text by creating **pseudo-labels** from the data itself.

$$\min_{\theta} \mathbb{E}_{x \sim D} \ell(f_{\theta}(\tilde{x}), x)$$

Causal Language Modeling (CLM): predict next token – GPT, GPT-2, GPT-3

Masked Language Modeling (MLM): predict masked tokens – BERT

Permutation/Span Prediction: T5

Contrastive Learning: SimCSE, CLIP (text-image)

Pretraining large language models (LLMs)

Transfer learning – fine-tuning for classification, NER, QA, summarization

Multimodal learning (text + images, speech, etc.)

Masked Language Modeling (MLM)

Randomly mask ~15% of input tokens.

Model predicts the original words from context.

Loss: cross-entropy between predicted and true tokens.

Masked Language Modeling (MLM)

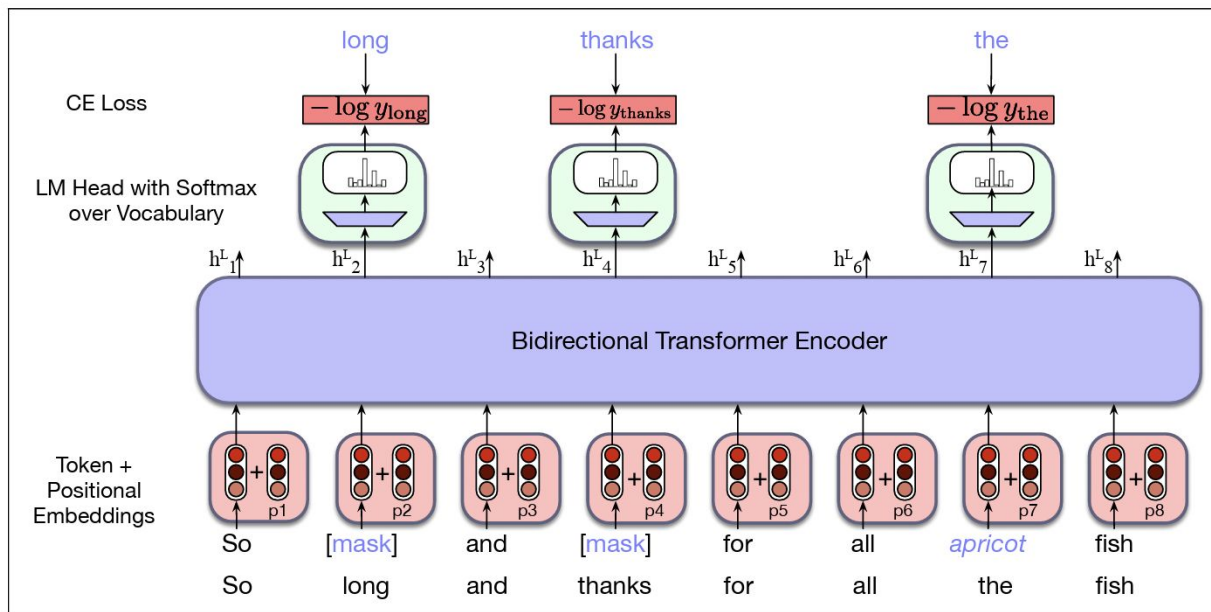


Figure 9.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

Reinforcement learning

A model (agent) **learns by trial and error**, tries an action, gets feedback (reward or penalty), improves its policy.

Intuition: instead just predicting the next word, try to **optimize long-term outcomes** (better translations, better answers)

Dialogue systems: learn to keep conversations useful.

Machine translation and Summarization: improve scores by using it as reward.

LLM alignment: Reinforcement Learning from Human Feedback (RLHF).

TODO:

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003).

[Latent Dirichlet Allocation. JMLR.](#)

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).

[Efficient Estimation of Word Representations in Vector Space](#)

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).

[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.](#)

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018).

[Improving Language Understanding by Generative Pre-Training. OpenAI.](#)

Raffel, C., Shazeer, N., Roberts, A., et al. (2020).

[Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer \(T5\).](#)

Gao, T., Yao, X., & Chen, D. (2021).

[SimCSE: Simple Contrastive Learning of Sentence Embeddings.](#)

LeCun, Y. (2022).

[Self-supervised learning: The dark matter of intelligence.](#)

Williams, R. J. (1992).

[Simple statistical gradient-following algorithms for connectionist reinforcement learning \(REINFORCE\).](#)

Bahdanau, D., Brakel, P., Xu, K., et al. (2017).

[An Actor-Critic Algorithm for Sequence Prediction.](#)

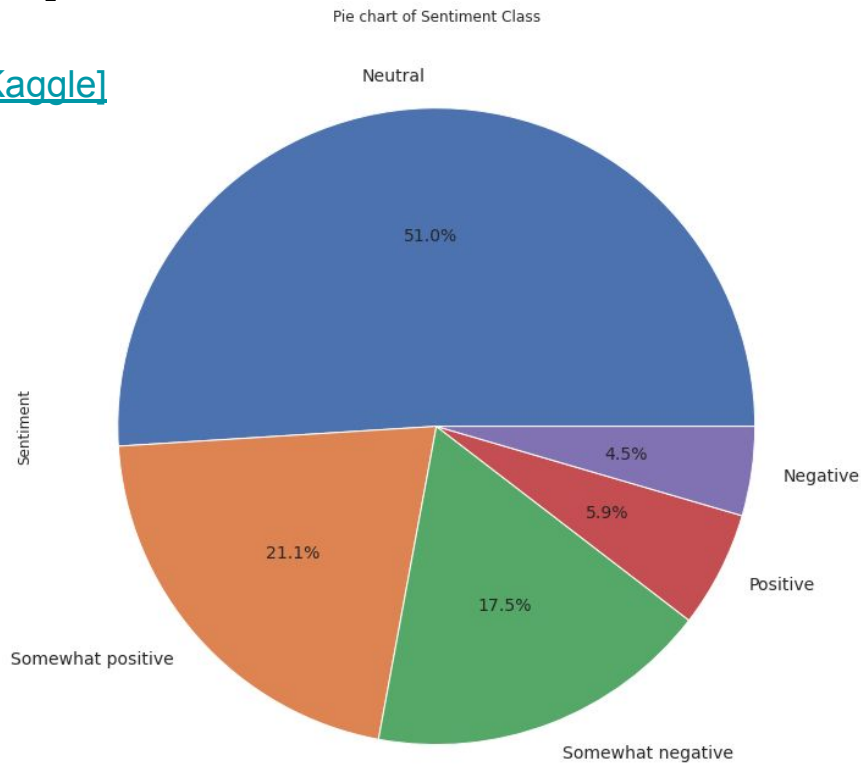
Ouyang, L., Wu, J., Jiang, X., et al. (2022).

[Training language models to follow instructions with human feedback. NeurIPS.](#)

Text Classification - Sentiment Analysis

Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

1. EDA



Text Classification - Sentiment Analysis

Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

1. EDA
2. Option 1. Classical ML:
 - a. **Feature engineering:** Bag-of-Words, TF-IDF, n-grams
 - b. **Models:** Logistic Regression, Naïve Bayes, SVM
3. Option 2. Deep Learning:
 - a. **Word embeddings:** Word2Vec, GloVe
 - b. **Neural models:** CNNs, RNNs (LSTM)
4. Option 3. Modern NLP:
 - a. **Pretrained language models:** BERT, RoBERTa, DistilBERT
 - b. **Fine-tuning:** add a classification head
5. Option 4. Alternatives:
 - a. **Use prompting and any available LLM**

Text Classification - Sentiment Analysis

Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

1. EDA
2. Option 1. Classical ML:
 - a. **Feature engineering:** Bag-of-Words, TF-IDF, n-grams
 - b. **Models:** Logistic Regression, Naïve Bayes, SVM
3. Option 2. Deep Learning:
 - a. **Word embeddings:** Word2Vec, GloVe
 - b. **Neural models:** CNNs, RNNs (LSTM)
4. Option 3. Modern NLP:
 - a. **Pretrained language models:** BERT, RoBERTa, DistilBERT
 - b. **Fine-tuning:** add a classification head
5. Option 4. Alternatives:
 - a. **Use prompting and any available LLM**

Text Classification - Sentiment Analysis

Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

ULMFiT introduced a **transfer learning approach**:

1. EDA
 2. Option 1. Classical ML:
 - a. **Feature engineering:** Bag-of-Words, TF-IDF, n-grams
 - b. **Models:** Logistic Regression, Naïve Bayes, SVM
 3. Option 2. Deep Learning:
 - a. **Word embeddings:** Word2Vec, GloVe
 - b. **Neural models:** CNNs, RNNs (LSTM)
 4. Option 3. Modern NLP:
 - a. **Pretrained language models:** BERT, RoBERTa, DistilBERT
 - b. **Fine-tuning:** add a classification head
 5. Option 4. Alternatives:
 - a. Use prompting and any available LLM
1. Pretrain a language model on a large general corpus (e.g., Wikipedia).
 2. Fine-tune the language model on domain data (e.g., movie reviews).
 3. Add a classifier head and fine-tune on the target task (e.g., sentiment analysis).

Howard, J., & Ruder, S. (2018).

[Universal Language Model Fine-tuning for Text Classification](#)

Text Classification - Sentiment Analysis

Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

1. EDA
2. Option 1. Classical ML:
 - a. **Feature engineering:** Bag-of-Words, TF-IDF, n-grams
 - b. **Models:** Logistic Regression, Naïve Bayes, SVM
3. Option 2. Deep Learning:
 - a. **Word embeddings:** Word2Vec, GloVe
 - b. **Neural models:** CNNs, RNNs (LSTM)
4. Option 3. Modern NLP:
 - a. **Pretrained language models:** BERT, RoBERTa, DistilBERT
 - b. **Fine-tuning:** add a classification head
5. Option 4. Alternatives:
 - a. **Use prompting and any available LLM**

Text Classification - Sentiment Analysis

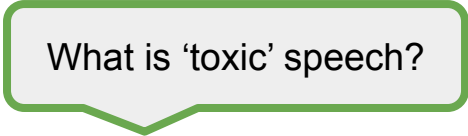
Example: [Sentiment Analysis on Movie Reviews \[Kaggle\]](#)

1. EDA
2. Option 1. Classical ML:
 - a. **Feature engineering:** Bag-of-Words, TF-IDF, n-grams
 - b. **Models:** Logistic Regression, Naïve Bayes, SVM
3. Option 2. Deep Learning:
 - a. **Word embeddings:** Word2Vec, GloVe
 - b. **Neural models:** CNNs, RNNs (LSTM)
4. Option 3. Modern NLP:
 - a. **Pretrained language models:** BERT, RoBERTa, DistilBERT
 - b. **Fine-tuning:** add a classification head
5. Option 4. Alternatives:
 - a. **Use prompting and any available LLM**

Pang, B., & Lee, L. (2008). *Opinion Mining and Sentiment Analysis*.

Text Classification - Toxicity/Hate speech detection

Example: [Toxic Comment Classification Challenge \[Kaggle\]](#)



What is 'toxic' speech?

Text Classification - Toxicity/Hate speech detection

Example: [Toxic Comment Classification Challenge \[Kaggle\]](#)

“Go back to your country 🙄”

What is ‘toxic’ speech?

Text Classification - Toxicity/Hate speech detection

Example: [Toxic Comment Classification Challenge \[Kaggle\]](#)

Hate speech can be subtle, implicit, or sarcastic.

most online comments are non-toxic

“Go back to your country 🙄”

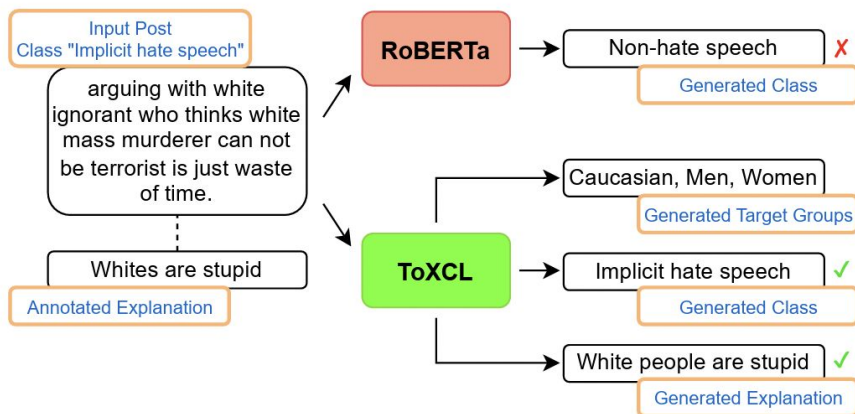
models may unfairly flag dialects as toxic

explicit (clear insults, slurs)

implicit (coded, indirect, or sarcastic language)

There may be pretrained and fine-tuned on toxic corpora models for example [hateBERT](#), [paper](#)

Text Classification - Toxicity/Hate speech detection



[ToXCL: A Unified Framework for Toxic Speech Detection and Explanation](#)
[Nhat M. Hoang et al.](#)

Figure 1: A sample input post and its ground truth explanation from the test set of Implicit Hate Corpus (ElSherief et al., 2021). The input post is fed into both models. The baseline model, RoBERTa, fails to detect the implicit toxic speech while our proposed **ToXCL** model successfully detects it and generates a toxic explanation that closely matches the ground truth explanation.

Text Classification - Toxicity/Hate speech detection

Target Group Generator (TG):

- Identifies which demographic/minority group is targeted (e.g., '*Caucasian, Women*').
- Built using a T5 encoder-decoder model.

Encoder-Decoder Model:

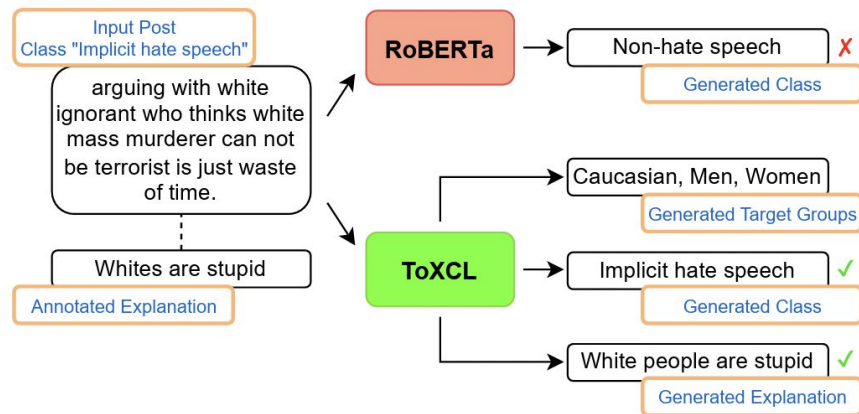
- Encoder:** does binary classification (toxic vs. non-toxic).
- Decoder:** generates explanations (only if toxic).

Teacher Classifier (TC):

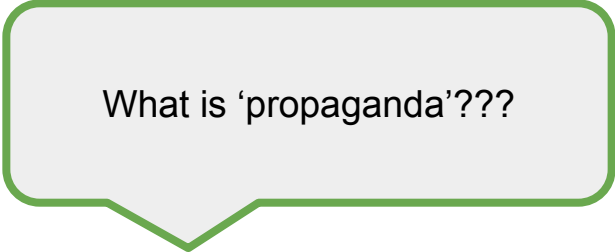
- A strong RoBERTa-Large model acts as a 'teacher'
- Its knowledge is distilled into the encoder classifier to improve detection.

Conditional Decoding (CD):

- Prevents generating explanations when text is **non-toxic** (forces output = [None]).

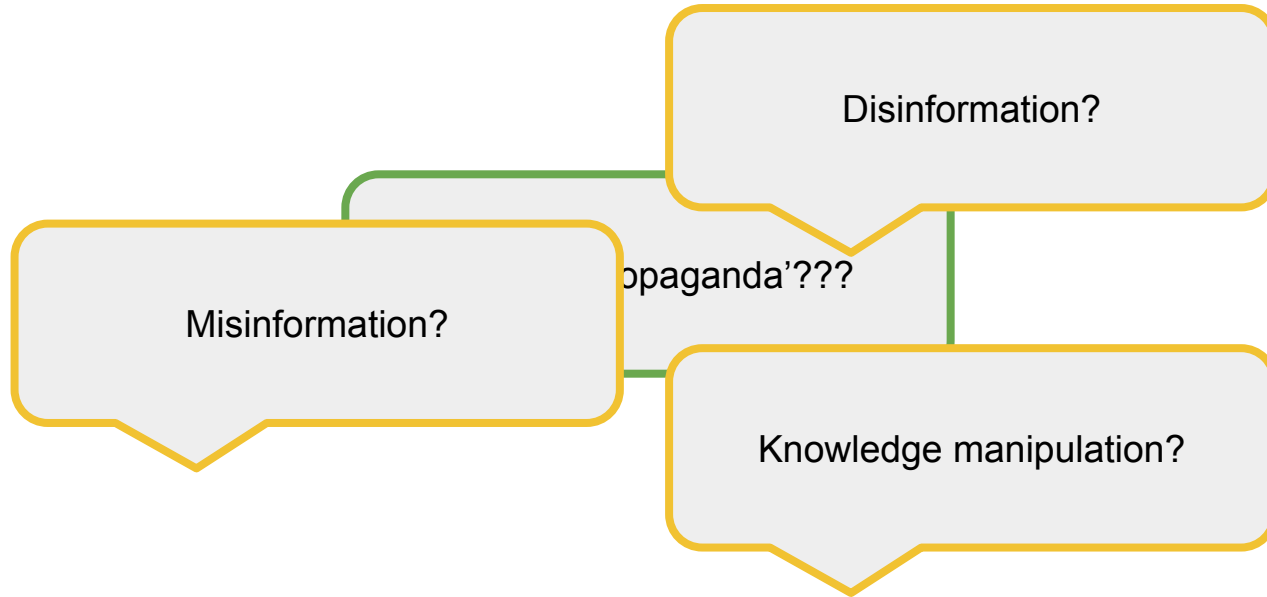


Text Classification - Propaganda detection



What is 'propaganda'???

Text Classification - Propaganda detection



Text Classification - Propaganda detection

Check News in One Click: NLP-Empowered Pro-Kremlin Propaganda Detection

Veronika Solopova, Viktoriia Herman,
Christoph Benz Müller, Tim Landgraf

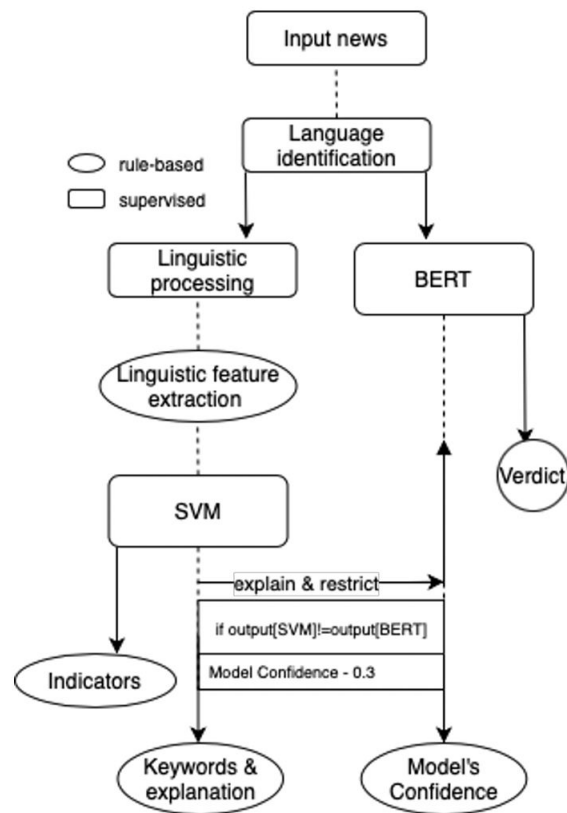


Figure 1: The figure illustrates the system's mock-up. The elliptical elements are rule-based reasoners while squared ones are trained models.

Text Classification - Vandalism detection

Fair multilingual vandalism detection system for Wikipedia

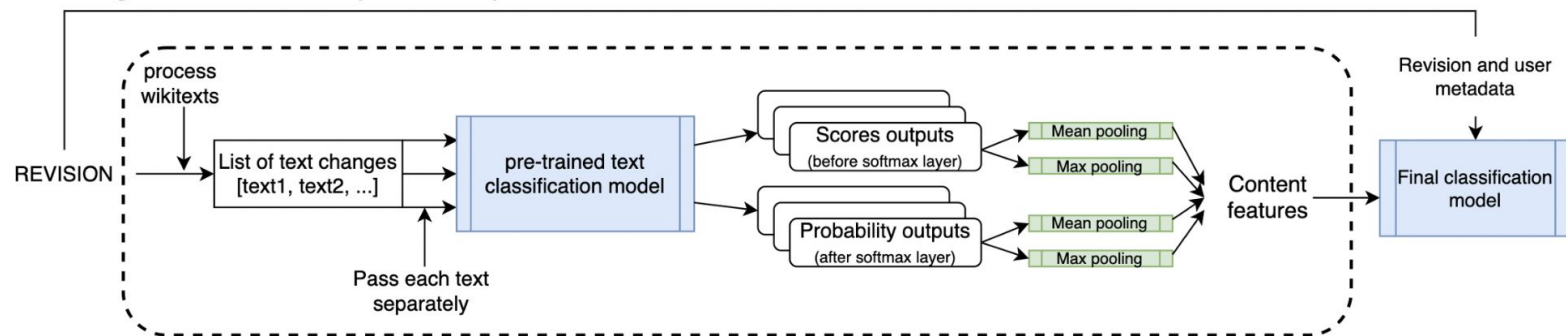


Figure 1: System inference logic schema.

[Fair multilingual vandalism detection system for Wikipedia](#)

[Mykola Trokhymovych et al.](#)

Text Classification - Vandalism detection

Fair multilingual vandalism detection system for Wikipedia

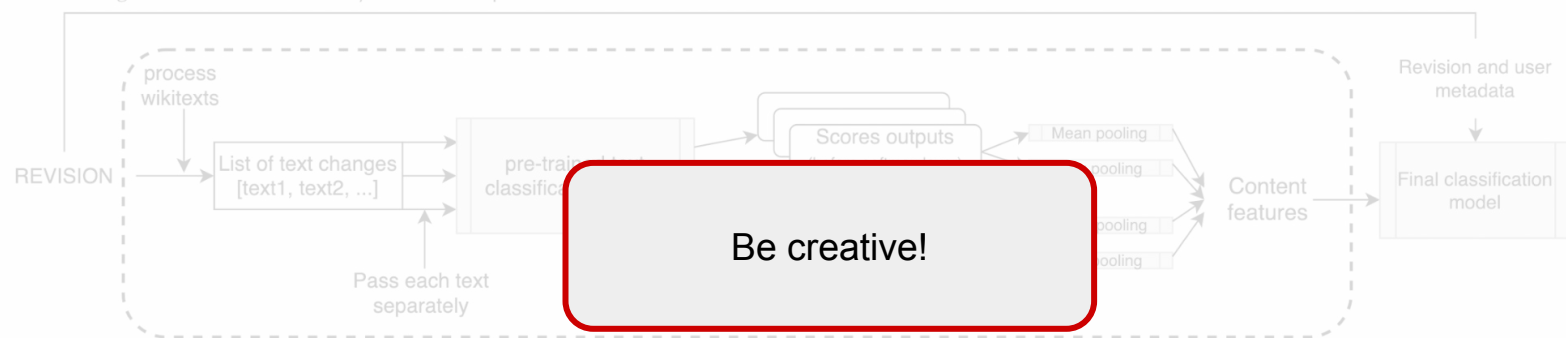


Figure 1: System inference logic schema.

Fair multilingual vandalism detection system for Wikipedia

Mykola Trokhymovych et al.

Text Classification - Manipulation detection

UNLP 2025 shared task

- Subtask 1 (**Technique classification**): given the text of a post, identify which manipulation techniques are used, if any. This is a multilabel classification problem; a single post can contain multiple techniques.
- Subtask 2 (**Span identification**): given the text of a post, identify the specific spans of manipulative text, regardless of the manipulation technique. This is a binary token-level classification task, focusing on pinpointing exactly where the manipulative content occurs.

Technique	Count
Loaded Language	4,932
Cherry Picking	1,280
Glittering Generalities	1,206
Euphoria	1,157
Cliché	1,158
FUD (Fear, Uncertainty, Doubt)	961
Appeal to Fear	750
Whataboutism	393
Bandwagon	393
Straw Man	345

Table 2: Frequency of manipulation techniques (a post may contain multiple techniques).

[source](#)

Text Classification - Manipulation detection

Is it easier when you have ground truth?

[RWFork: Russian Wikipedia Fork Edits Dataset](#)

[Characterizing Knowledge Manipulation in a Russian Wikipedia Fork](#)

Mykola Trokhymovych, Oleksandr Kosovan,
Nathan Forrester, et al. [2025]

Text Classification - Manipulation detection

Is it easier when you have ground truth?

[RWFork: Russian Wikipedia Fork Edits Dataset](#)

[Characterizing Knowledge Manipulation in a Russian Wikipedia Fork](#)

Mykola Trokhymovych, Oleksandr Kosovan,
Nathan Forrester, et al. [2025]

Wow! You have some examples of
manipulative texts now.

Design knowledge manipulation
detection system.

What components it must have? Your
steps?

Text Classification - Summary

Option 3. Modern NLP:

- a. **Pretrained language models:**
BERT, RoBERTa, DistilBERT
- b. **Fine-tuning:** add a classification head

Text Classification - Summary

Option 3. Modern NLP:

- a. **Pretrained language models:**
BERT, RoBERTa, DistilBERT
- b. **Fine-tuning:** add a classification head

“Encoder-only models produce an encoding for each input token but generally aren’t used to produce running text by decoding/sampling.

That’s an important point: masked language models are not used for generation. They are generally instead used for interpretative tasks.”

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft).

Encoder-only

1. **Input tokens → processed through stacked self-attention + feedforward layers in the encoder.**
2. **The model outputs contextualized embeddings for each token.**
3. **No autoregressive decoding - just a contextual representation.**
4. **Examples: BERT, RoBERTa, DistilBERT, ALBERT**

Decoder-only

1. Uses only the decoder stack of the original Transformer.
2. Self-attention is causal (masked) so the model can't peek at future tokens.
3. Trained autoregressively: predict the next token given previous ones.
4. Examples: GPT family (GPT-2, GPT-3, GPT-4, GPT-4o, ChatGPT), LLaMA, Mistral, Falcon, etc.

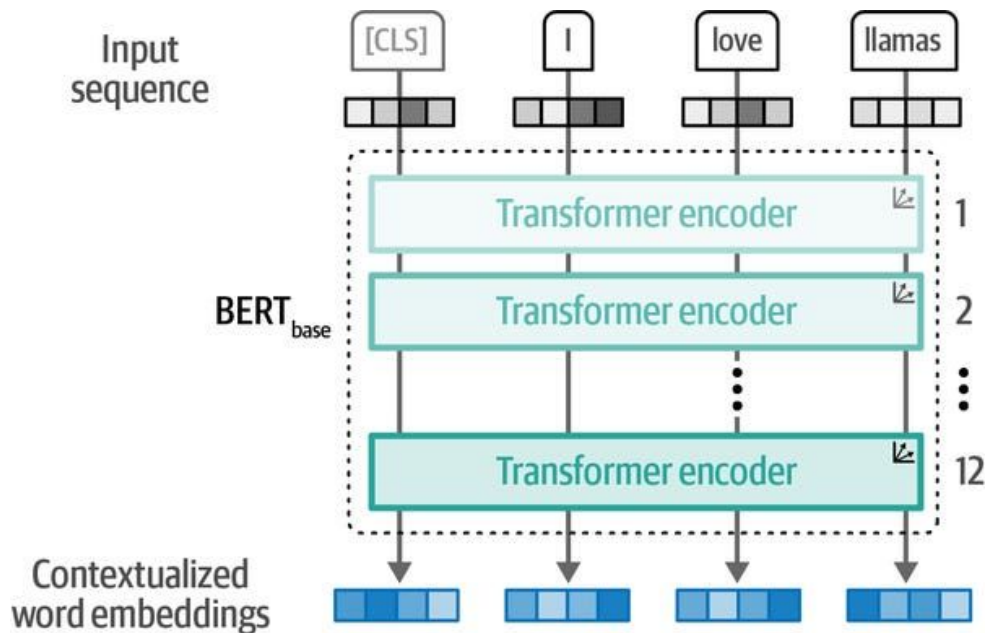
Encoder-decoder

1. Encoder: Processes the full input sequence into contextual embeddings.
2. Decoder: Generates output tokens autoregressively, attending both to previously generated tokens and the encoder's representations.
3. This allows mapping one sequence to another.
4. Examples: T5 (Text-to-Text Transfer Transformer), BART, mBART, MarianMT (translation).

Text Classification - Summary

Option 3. Modern NLP:

- a. **Pretrained language models:**
BERT, RoBERTa, DistilBERT
- b. **Fine-tuning:** add a classification head



Sequence-to-Sequence (Seq2Seq) problems

Machine Translation

Summarization

QA (abstractive)

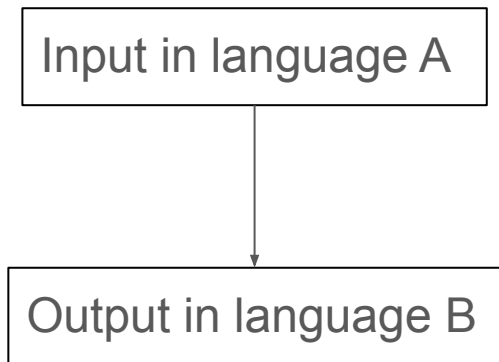
Input in language A



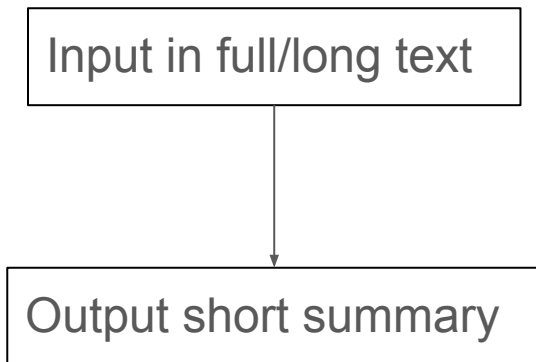
Output in language B

Sequence-to-Sequence (Seq2Seq) problems

Machine Translation



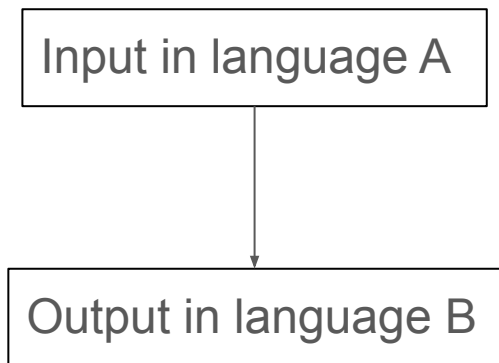
Summarization



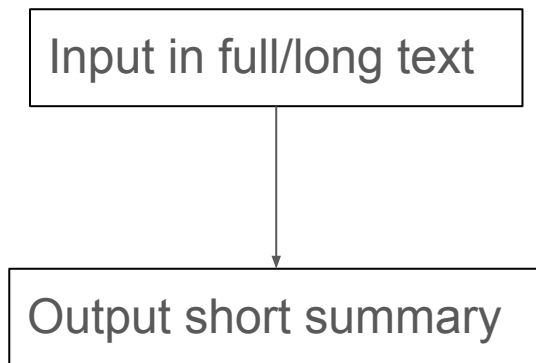
QA (abstractive)

Sequence-to-Sequence (Seq2Seq) problems

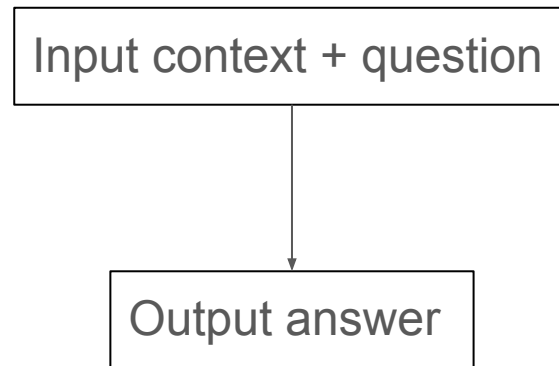
Machine Translation



Summarization



QA (abstractive)



Encoder-only

1. Input tokens → processed through stacked self-attention + feedforward layers in the encoder.
2. The model outputs contextualized embeddings for each token.
3. No autoregressive decoding - just a contextual representation.
4. Examples: BERT, RoBERTa, DistilBERT, ALBERT

Decoder-only

1. Uses only the decoder stack of the original Transformer.
2. Self-attention is causal (masked) so the model can't peek at future tokens.
3. Trained autoregressively: predict the next token given previous ones.
4. Examples: GPT family (GPT-2, GPT-3, GPT-4, GPT-4o, ChatGPT), LLaMA, Mistral, Falcon, etc.

Encoder-decoder

1. **Encoder: Processes the full input sequence into contextual embeddings.**
2. **Decoder: Generates output tokens autoregressively, attending both to previously generated tokens and the encoder's representations.**
3. **This allows mapping one sequence to another.**
4. **Examples: T5 (Text-to-Text Transfer Transformer), BART, mBART, MarianMT (translation).**

Sequence-to-Sequence (Seq2Seq) problems

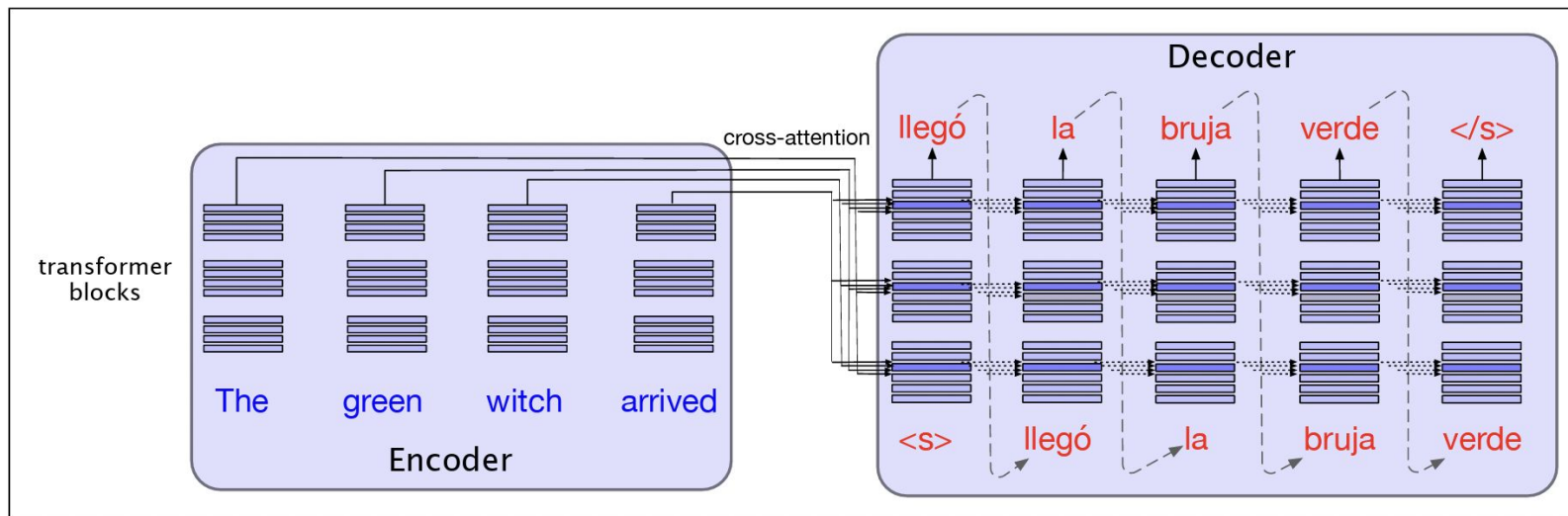


Figure 12.5 The encoder-decoder transformer architecture for machine translation. The encoder uses the transformer blocks we saw in Chapter 8, while the decoder uses a more powerful block with an extra **cross-attention** layer that can attend to all the encoder words. We'll see this in more detail in the next section.

Machine Translations

Parallel corpora:

FLORES – FLORES is a benchmark dataset for machine translation between English and low-resource languages.

```
{
  'id': 1,
  'URL':
'https://en.wikinews.org/wiki/Scientists_say_new_medical_diagnostic_chip_can_sort_cells_anywhere_
with_an_inkjet',
  'domain': 'wikinews',
  'topic': 'health',
  'has_image': 0,
  'has_hyperlink': 0,
  'sentence_eng_Latn': 'On Monday, scientists from the Stanford University School of Medicine
announced the invention of a new diagnostic tool that can sort cells by type: a tiny printable chip that
can be manufactured using standard inkjet printers for possibly about one U.S. cent each.',
  'sentence_ukr_Cyrl': 'У понеділок, науковці зі Школи медицини Стенфордського університету
оголосили про винайдення нового діагностичного інструменту, що може сортувати клітини за їх
видами: це малесенький друкований чіп, який можна виготовити за допомогою стандартних
променевих принтерів десь по одному центу США за штуку.'
}
```

Machine Translations

Parallel corpora:

[FLORES](#) - FLORES is a benchmark dataset for machine translation between English and low-resource languages.

[EuroParl](#) - A Parallel Corpus for Statistical Machine Translation, *Philipp Koehn*, MT Summit 2005.

[OPUSMT](#) - [OPUS-MT a project that focuses on the development of free resources and tools for machine translation](#)

Summarization

Long input, short/er output

[CNN / DailyMail](#) - article - highlights pairs

[XSum dataset](#) - Extreme Summarization (XSum)

Dataset. (document/article as input - only summary as output)

Question Answering

SQuAD **Stanford Question Answering Dataset** (SQuAD) is a reading comprehension dataset, consisting of questions posed by crowdworkers on a set of Wikipedia articles, where the answer to every question is a segment of text, or *span*, from the corresponding reading passage, or the question might be unanswerable.

Natural Questions Natural Questions (NQ) contains real user questions issued to Google search, and answers found from Wikipedia by annotators. NQ is designed for the training and evaluation of automatic question answering systems.

```
{
  "example_id": 3902,
  "document_url": "http://wikipedia.org/en/strings"
  "question_text": "what is a string",
  "document_text": "<P> A string is a list of characters in order . </P>",
  "annotations": [{
    "long_answer": { "start_token": 0, "end_token": 12 },
    "short_answers": [{ "start_token": 5, "end_token": 8 }],
    "yes_no_answer": "NONE",
  }],
  "long_answer_candidates": [
    { "start_token": 0, "end_token": 12, "top_level": True }
  ]
}
```

Question Answering

SQuAD **Stanford Question Answering Dataset** (SQuAD) is a reading comprehension dataset, consisting of questions posed by crowdworkers on a set of Wikipedia articles, where the answer to every question is a segment of text, or *span*, from the corresponding reading passage, or the question might be unanswerable.

Natural Questions Natural Questions (NQ) contains real user questions issued to Google search, and answers found from Wikipedia by annotators. NQ is designed for the training and evaluation of automatic question answering systems.

Extractive QA

Abstractive QA

Open-domain QA, Conversational QA

Question Answering

SQuAD Stanford Question Answering Dataset

(SQuAD) is a reading comprehension dataset, consisting of questions posed by crowdworkers on a set of Wikipedia articles, where the answer to every question is a segment of text, or *span*, from the corresponding reading passage. If the question might be unanswerable, the answer span is the empty string.

Extractive QA

Natural Questions

contains real user questions and answers found on the web. It is designed for the training and evaluation of automatic question answering systems.

```
{"context": "Architecturally, the school has a Catholic character. ...",  
"question": "To whom did the Virgin Mary allegedly appear in 1858 in Lourdes  
France?",  
"answers": {"text": ["Saint Bernadette Soubirous"],  
"answer_start": [515]}}
```

nal QA

TODO:

1. [BLEU](#) (original paper)
2. [Sequence to Sequence Learning with Neural Networks](#) (original seq2seq NMT paper)
3. [Sequence Transduction with Recurrent Neural Networks](#) (early seq2seq speech recognition paper)
4. [Neural Machine Translation by Jointly Learning to Align and Translate](#) (original seq2seq+attention paper)
5. [Massive Exploration of Neural Machine Translation Architectures](#) (practical advice for hyperparameter choices)
6. [Achieving Open Vocabulary Neural Machine Translation with Hybrid Word-Character Models](#)
7. [Revisiting Character-Based Neural Machine Translation with Capacity and Compression](#)

Encoder-only

1. Input tokens → processed through stacked self-attention + feedforward layers in the encoder.
2. The model outputs contextualized embeddings for each token.
3. No autoregressive decoding - just a contextual representation.
4. Examples: BERT, RoBERTa, DistilBERT, ALBERT

Decoder-only

1. **Uses only the decoder stack of the original Transformer.**
2. **Self-attention is causal (masked) so the model can't peek at future tokens.**
3. **Trained autoregressively: predict the next token given previous ones.**
4. **Examples: GPT family (GPT-2, GPT-3, GPT-4, GPT-4o, ChatGPT), LLaMA, Mistral, Falcon, etc.**

Encoder-decoder

1. Encoder: Processes the full input sequence into contextual embeddings.
2. Decoder: Generates output tokens autoregressively, attending both to previously generated tokens and the encoder's representations.
3. This allows mapping one sequence to another.
4. Examples: T5 (Text-to-Text Transfer Transformer), BART, mBART, MarianMT (translation).

Text Generation

Decoder-Only Models (GPT, LLaMA, etc.) Causal LM

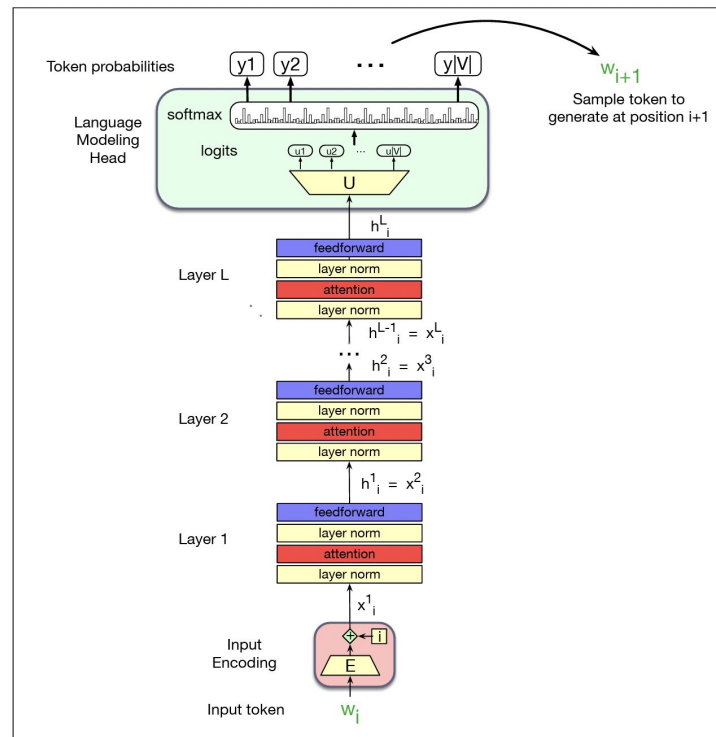
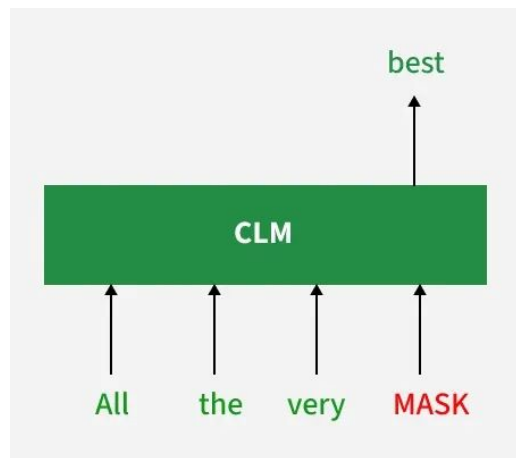


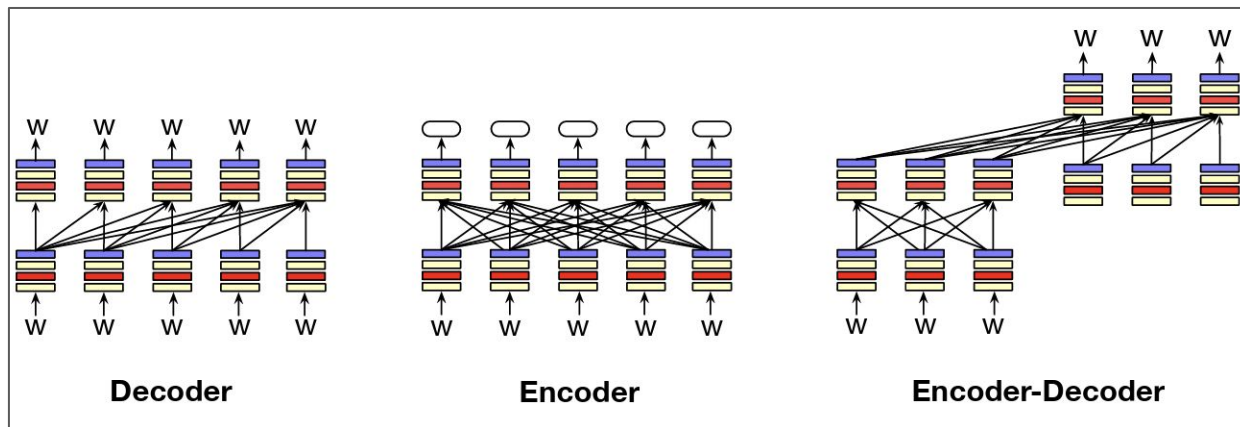
Figure 8.15 A transformer language model (decoder-only), stacking transformer blocks and mapping from an input token w_i to a predicted next token w_{i+1} .

Text Generation

OpenAssistant dataset

```
{
  "message_tree_id": "14fbb664-a620-45ce-bee4-7c519b16a793",
  "tree_state": "ready_for_export",
  "prompt": {
    "message_id": "14fbb664-a620-45ce-bee4-7c519b16a793",
    "text": "Why can't we divide by 0? (..)",
    "role": "prompter",
    "lang": "en",
    "replies": [
      {
        "message_id": "894d30b6-56b4-4605-a504-89dd15d4d1c8",
        "text": "The reason we cannot divide by zero is because (..)",
        "role": "assistant",
        "lang": "en",
        "replies": [
          // ...
        ]
      },
    ]
  },
  {
    "message_id": "84d0913b-0fd9-4508-8ef5-205626a7039d",
    "text": "The reason that the result of a division by zero is (..)",
    "role": "assistant",
    "lang": "en",
    "replies": [
```

Summary





Q&A



Homework
questions?

Q&A

	Topic	Pts.	Start	Deadline
HW1	Text Classification	10	19.09	02.10
HW 2	Translation Task	15	10.10	19.10
HW 3	LLM Finetuning	15	03.11	16.11
HW 4	Retrieval-Augmented Generation (RAG)	15	24.11	07.12
Online Test 1	Week 1 - 5	5	03.10	08.10
Online Test 2	Week 6 - 10	5	07.11	10.11
Online Test 3	Week 11 - 15	5	08.12	10.12
Course Project - Mid report		5	29.09	07.11
Course Project - Defence		25	29.09	12.12

Final project
is coming

TODO:



Final project is coming

TODO:

Divide by teams by 3
or 2 people

Think of a project idea (list of options
will be provided additionally ...soon)

